

Deep Agents reference

v0.3.0 license MIT downloads 561k

Welcome to the [Deep Agents](#) reference documentation!

Work in progress

This page is a work in progress, and we appreciate your patience as we continue to expand and improve the content.

mod deepagents

DeepAgents package.

FUNCTION	DESCRIPTION
<code>create_deep_agent</code>	Create a deep agent.

class FilesystemMiddleware

Bases: `AgentMiddleware`

Middleware for providing filesystem and optional execution tools to an agent.

This middleware adds filesystem tools to the agent: `ls`, `read_file`, `write_file`, `edit_file`, `glob`, and `grep`. Files can be stored using any backend that implements the `BackendProtocol`.

If the backend implements `SandboxBackendProtocol`, an `execute` tool is also added for running shell commands.

PARAMETER	DESCRIPTION
<code>backend</code>	Backend for file storage and optional execution. If not provided, defaults to <code>StateBackend</code> (ephemeral storage)

PARAMETER	DESCRIPTION
	in agent state). For persistent storage or hybrid setups, use CompositeBackend with custom routes. For execution support, use a backend that implements SandboxBackendProtocol.
	TYPE: <code>BACKEND_TYPES</code> <code>None</code> DEFAULT: <code>None</code>
<code>system_prompt</code>	Optional custom system prompt override.
	TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>custom_tool_descriptions</code>	Optional custom tool descriptions override.
	TYPE: <code>dict[str, str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>tool_token_limit_before_evict</code>	Optional token limit before evicting a tool result to the filesystem.
	TYPE: <code>int</code> <code>None</code> DEFAULT: <code>20000</code>



Example



```
from deepagents.middleware.filesystem import FilesystemMiddleware
from deepagents.backends import StateBackend, StoreBackend, CompositeBackend
from langchain.agents import create_agent

# Ephemeral storage only (default, no execution)
agent = create_agent(middleware=[FilesystemMiddleware()])

# With hybrid storage (ephemeral + persistent /memories/)
backend = CompositeBackend(default=StateBackend(), routes={"/memories/":
StoreBackend()})
agent = create_agent(middleware=[FilesystemMiddleware(backend=backend)])

# With sandbox backend (supports execution)
from my_sandbox import DockerSandboxBackend

sandbox = DockerSandboxBackend(container_id="my-container")
agent = create_agent(middleware=[FilesystemMiddleware(backend=sandbox)])
```

METHOD	DESCRIPTION
<code>before_agent</code>	Logic to run before the agent execution starts.
<code>abefore_agent</code>	Async logic to run before the agent execution starts.
<code>before_model</code>	Logic to run before the model is called.
<code>abefore_model</code>	Async logic to run before the model is called.
<code>after_model</code>	Logic to run after the model is called.
<code>aafter_model</code>	Async logic to run after the model is called.
<code>after_agent</code>	Logic to run after the agent execution completes.
<code>aafter_agent</code>	Async logic to run after the agent execution completes.
<code>__init__</code>	Initialize the filesystem middleware.
<code>wrap_model_call</code>	Update the system prompt and filter tools based on backend capabilities.
<code>awrap_model_call</code>	(async) Update the system prompt and filter tools based on backend capabilities.
<code>wrap_tool_call</code>	Check the size of the tool call result and evict to filesystem if too large.
<code>awrap_tool_call</code>	(async) Check the size of the tool call result and evict to filesystem if too large.

attr **name** property

```
name: str
```



The name of the middleware instance.

Defaults to the class name, but can be overridden for custom naming.

attr **state_schema** class-attribute instance-attribute

```
state_schema = FileSystemState
```

The schema for state passed to the middleware nodes.

attr **tools** instance-attribute

```
tools = _get_filesystem_tools(backend, custom_tool_descriptions)
```

Additional tools registered by the middleware.

meth **before_agent**

```
before_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run before the agent execution starts.

Async version is `abefore_agent`

meth **abefore_agent** async

```
abefore_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run before the agent execution starts.

meth **before_model**

```
before_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run before the model is called.

Async version is `abefore_model`

meth **abefore_model** async

```
abefore_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run before the model is called.

meth **after_model**

```
after_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run after the model is called.

Async version is `aafter_model`

```
meth aafter_model async
```

```
aafter_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run after the model is called.

```
meth after_agent
```

```
after_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run after the agent execution completes.

```
meth aafter_agent async
```

```
aafter_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run after the agent execution completes.

```
meth __init__
```

```
__init__(
    *,
    backend: BACKEND_TYPES | None = None,
    system_prompt: str | None = None,
    custom_tool_descriptions: dict[str, str] | None = None,
    tool_token_limit_before_evict: int | None = 20000,
) -> None
```

Initialize the filesystem middleware.

PARAMETER	DESCRIPTION
<code>backend</code>	Backend for file storage and optional execution, or a factory callable. Defaults to StateBackend if not provided.
	TYPE: <code>BACKEND_TYPES None</code> DEFAULT: <code>None</code>

PARAMETER	DESCRIPTION
<code>system_prompt</code>	Optional custom system prompt override. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>custom_tool_descriptions</code>	Optional custom tool descriptions override. TYPE: <code>dict[str, str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>tool_token_limit_before_evict</code>	Optional token limit before evicting a tool result to the filesystem. TYPE: <code>int</code> <code>None</code> DEFAULT: <code>20000</code>

meth `wrap_model_call`

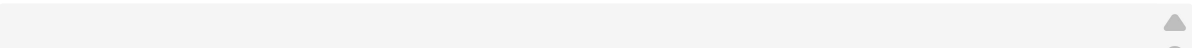
```
wrap_model_call(  
    request: ModelRequest, handler: Callable[[ModelRequest], ModelResponse]  
) -> ModelResponse
```

Update the system prompt and filter tools based on backend capabilities.

PARAMETER	DESCRIPTION
<code>request</code>	The model request being processed. TYPE: <code>ModelRequest</code>
<code>handler</code>	The handler function to call with the modified request. TYPE: <code>Callable[[ModelRequest], ModelResponse]</code>

RETURNS	DESCRIPTION
<code>ModelResponse</code>	The model response from the handler.

meth `awrap_model_call` `async`



```
awrap_model_call(  
    request: ModelRequest, handler: Callable[[ModelRequest],  
    Awaitable[ModelResponse]]  
    ) -> ModelResponse
```

(async) Update the system prompt and filter tools based on backend capabilities.

PARAMETER	DESCRIPTION
<code>request</code>	The model request being processed. TYPE: <code>ModelRequest</code>
<code>handler</code>	The handler function to call with the modified request. TYPE: <code>Callable[[ModelRequest], Awaitable[ModelResponse]]</code>

RETURNS	DESCRIPTION
<code>ModelResponse</code>	The model response from the handler.

meth `wrap_tool_call`

```
wrap_tool_call(  
    request: ToolCallRequest,  
    handler: Callable[[ToolCallRequest], ToolMessage | Command],  
    ) -> ToolMessage | Command
```

Check the size of the tool call result and evict to filesystem if too large.

PARAMETER	DESCRIPTION
<code>request</code>	The tool call request being processed. TYPE: <code>ToolCallRequest</code>
<code>handler</code>	The handler function to call with the modified request. TYPE: <code>Callable[[ToolCallRequest], ToolMessage Command]</code>

RETURNS	DESCRIPTION
<code>ToolMessage</code> <code>Command</code>	The raw <code>ToolMessage</code> , or a pseudo tool message with the <code>ToolResult</code> in state.

meth `awrap_tool_call` async

```
awrap_tool_call(  
    request: ToolCallRequest,  
    handler: Callable[[ToolCallRequest], Awaitable[ToolMessage | Command]],  
) -> ToolMessage | Command
```

(async)Check the size of the tool call result and evict to filesystem if too large.

PARAMETER	DESCRIPTION
<code>request</code>	The tool call request being processed. TYPE: <code>ToolCallRequest</code>
<code>handler</code>	The handler function to call with the modified request. TYPE: <code>Callable[[ToolCallRequest], Awaitable[ToolMessage Command]]</code>

RETURNS	DESCRIPTION
<code>ToolMessage</code> <code>Command</code>	The raw <code>ToolMessage</code> , or a pseudo tool message with the <code>ToolResult</code> in state.

class `CompiledSubAgent`

Bases: `TypedDict`

A pre-compiled agent spec.

attr `name` instance-attribute

```
name: str
```

The name of the agent.

attr **description** instance-attribute

```
description: str
```



The description of the agent.

attr **runnable** instance-attribute

```
runnable: Runnable
```



The Runnable to use for the agent.

class SubAgent

Bases: `TypedDict`

Specification for an agent.

When specifying custom agents, the `default_middleware` from `SubAgentMiddleware` will be applied first, followed by any `middleware` specified in this spec. To use only custom middleware without the defaults, pass `default_middleware=[]` to `SubAgentMiddleware`.

attr **name** instance-attribute

```
name: str
```



The name of the agent.

attr **description** instance-attribute

```
description: str
```



The description of the agent.

attr **system_prompt** instance-attribute

```
system_prompt: str
```



The system prompt to use for the agent.

attr **tools** instance-attribute

```
tools: Sequence[BaseTool | Callable | dict[str, Any]]
```

The tools to use for the agent.

attr **model** instance-attribute

```
model: NotRequired[str | BaseChatModel]
```

The model for the agent. Defaults to `default_model`.

attr **middleware** instance-attribute

```
middleware: NotRequired[list[AgentMiddleware]]
```

Additional middleware to append after `default_middleware`.

attr **interrupt_on** instance-attribute

```
interrupt_on: NotRequired[dict[str, bool | InterruptOnConfig]]
```

The tool configs to use for the agent.

class SubAgentMiddleware

Bases: `AgentMiddleware`

Middleware for providing subagents to an agent via a `task` tool.

This middleware adds a `task` tool to the agent that can be used to invoke subagents.

Subagents are useful for handling complex tasks that require multiple steps, or tasks that require a lot of context to resolve.

A chief benefit of subagents is that they can handle multi-step tasks, and then return a clean, concise response to the main agent.

Subagents are also great for different domains of expertise that require a narrower subset of tools and focus.

This middleware comes with a default general-purpose subagent that can be used to handle the same tasks as the main agent, but with isolated context.

PARAMETER	DESCRIPTION
<code>default_model</code>	The model to use for subagents. Can be a <code>LanguageModelLike</code> or a dict for <code>init_chat_model</code>. TYPE: <code>str</code> <code>BaseChatModel</code>
<code>default_tools</code>	The tools to use for the default general-purpose subagent. TYPE: <code>Sequence[BaseTool Callable dict[str, Any]]</code> <code>None</code> DEFAULT: <code>None</code>
<code>default_middleware</code>	Default middleware to apply to all subagents. If <code>None</code> (default), no default middleware is applied. Pass a list to specify custom middleware. TYPE: <code>list[AgentMiddleware]</code> <code>None</code> DEFAULT: <code>None</code>
<code>default_interrupt_on</code>	The tool configs to use for the default general-purpose subagent. These are also the fallback for any subagents that don't specify their own tool configs. TYPE: <code>dict[str, bool InterruptOnConfig]</code> <code>None</code> DEFAULT: <code>None</code>
<code>subagents</code>	A list of additional subagents to provide to the agent. TYPE: <code>list[SubAgent CompiledSubAgent]</code> <code>None</code> DEFAULT: <code>None</code>
<code>system_prompt</code>	Full system prompt override. When provided, completely replaces the agent's system prompt. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>TASK_SYSTEM_PROMPT</code>
<code>general_purpose_agent</code>	Whether to include the general-purpose agent. Defaults to <code>True</code>. TYPE: <code>bool</code> DEFAULT: <code>True</code>
<code>task_description</code>	Custom description for the task tool. If <code>None</code>, uses the default description template.

PARAMETER	DESCRIPTION
	TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>

Example

```
from langchain.agents.middleware.subagents import SubAgentMiddleware
from langchain.agents import create_agent

# Basic usage with defaults (no default middleware)
agent = create_agent(
    "openai:gpt-4o",
    middleware=[
        SubAgentMiddleware(
            default_model="openai:gpt-4o",
            subagents=[],
        )
    ],
)

# Add custom middleware to subagents
agent = create_agent(
    "openai:gpt-4o",
    middleware=[
        SubAgentMiddleware(
            default_model="openai:gpt-4o",
            default_middleware=[TodoListMiddleware()],
            subagents=[],
        )
    ],
)
```

METHOD	DESCRIPTION
<code>before_agent</code>	Logic to run before the agent execution starts.
<code>abefore_agent</code>	Async logic to run before the agent execution starts.
<code>before_model</code>	Logic to run before the model is called.
<code>abefore_model</code>	Async logic to run before the model is called.
<code>after_model</code>	Logic to run after the model is called.

METHOD	DESCRIPTION
<code>aafter_model</code>	Async logic to run after the model is called.
<code>after_agent</code>	Logic to run after the agent execution completes.
<code>aafter_agent</code>	Async logic to run after the agent execution completes.
<code>wrap_tool_call</code>	Intercept tool execution for retries, monitoring, or modification.
<code>awrap_tool_call</code>	Intercept and control async tool execution via handler callback.
<code>__init__</code>	Initialize the SubAgentMiddleware.
<code>wrap_model_call</code>	Update the system prompt to include instructions on using subagents.
<code>awrap_model_call</code>	(async) Update the system prompt to include instructions on using subagents.

attr **state_schema** class-attribute instance-attribute

```
state_schema: type[StateT] = cast('type[StateT]', AgentState)
```

The schema for state passed to the middleware nodes.

attr **name** property

```
name: str
```

The name of the middleware instance.

Defaults to the class name, but can be overridden for custom naming.

attr **tools** instance-attribute

```
tools = [task_tool]
```

Additional tools registered by the middleware.

meth before_agent

```
before_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run before the agent execution starts.

Async version is `abefore_agent`

meth abefore_agent `async`

```
abefore_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run before the agent execution starts.

meth before_model

```
before_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run before the model is called.

Async version is `abefore_model`

meth abefore_model `async`

```
abefore_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run before the model is called.

meth after_model

```
after_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run after the model is called.

Async version is `aafter_model`

meth aafter_model `async`

```
aafter_model(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run after the model is called.

meth after_agent

```
after_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Logic to run after the agent execution completes.

```
meth aafter_agent async
```

```
aafter_agent(state: StateT, runtime: Runtime[ContextT]) -> dict[str, Any] | None
```

Async logic to run after the agent execution completes.

```
meth wrap_tool_call
```

```
wrap_tool_call(
    request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], ToolMessage | Command],
) -> ToolMessage | Command
```

Intercept tool execution for retries, monitoring, or modification.

Async version is `awrap_tool_call`

Multiple middleware compose automatically (first defined = outermost).

Exceptions propagate unless `handle_tool_errors` is configured on `ToolNode`.

PARAMETER	DESCRIPTION
<code>request</code>	Tool call request with call <code>dict</code>, <code>BaseTool</code>, <code>state</code>, and <code>runtime</code>. Access state via <code>request.state</code> and runtime via <code>request.runtime</code>. TYPE: <code>ToolCallRequest</code>
<code>handler</code>	<code>Callable</code> to execute the tool (can be called multiple times). TYPE: <code>Callable[[ToolCallRequest], ToolMessage Command]</code>

RETURNS	DESCRIPTION
<code>ToolMessage</code> <code>Command</code>	<code>ToolMessage</code> or <code>Command</code> (the final result).

The handler `Callable` can be invoked multiple times for retry logic.

Each call to handler is independent and stateless.

Examples:

Modify request before execution

```
def wrap_tool_call(self, request, handler):
    modified_call = {
        **request.tool_call,
        "args": {
            **request.tool_call["args"],
            "value": request.tool_call["args"]["value"] * 2,
        },
    },
    request = request.override(tool_call=modified_call)
    return handler(request)
```

Retry on error (call handler multiple times)

```
def wrap_tool_call(self, request, handler):
    for attempt in range(3):
        try:
            result = handler(request)
            if is_valid(result):
                return result
        except Exception:
            if attempt == 2:
                raise
    return result
```

Conditional retry based on response

```
def wrap_tool_call(self, request, handler):
    for attempt in range(3):
        result = handler(request)
        if isinstance(result, ToolMessage) and result.status != "error":
            return result
        if attempt < 2:
            continue
    return result
```

meth `awrap_tool_call` `async`

`awrap_tool_call(`

```
request: ToolCallRequest,  
handler: Callable[[ToolCallRequest], Awaitable[ToolMessage | Command]],  
) -> ToolMessage | Command
```

Intercept and control async tool execution via handler callback.

The handler callback executes the tool call and returns a `ToolMessage` or `Command`. Middleware can call the handler multiple times for retry logic, skip calling it to short-circuit, or modify the request/response. Multiple middleware compose with first in list as outermost layer.

PARAMETER	DESCRIPTION
<code>request</code>	Tool call request with call <code>dict</code>, <code>BaseTool</code>, <code>state</code>, and <code>runtime</code>. Access state via <code>request.state</code> and runtime via <code>request.runtime</code>. TYPE: <code>ToolCallRequest</code>
<code>handler</code>	Async callable to execute the tool and returns <code>ToolMessage</code> or <code>Command</code>. Call this to execute the tool. Can be called multiple times for retry logic. Can skip calling it to short-circuit. TYPE: <code>Callable[[ToolCallRequest], Awaitable[ToolMessage Command]]</code>

RETURNS	DESCRIPTION
<code>ToolMessage Command</code>	<code>ToolMessage</code> or <code>Command</code> (the final result).

The handler `Callable` can be invoked multiple times for retry logic.

Each call to handler is independent and stateless.

Examples:

Async retry on error

```

async def awrap_tool_call(self, request, handler):
    for attempt in range(3):
        try:
            result = await handler(request)
            if is_valid(result):
                return result
        except Exception:
            if attempt == 2:
                raise
    return result

```

```

async def awrap_tool_call(self, request, handler):
    if cached := await get_cache_async(request):
        return ToolMessage(content=cached, tool_call_id=request.tool_call["id"])
    result = await handler(request)
    await save_cache_async(request, result)
    return result

```

meth `__init__`

```

__init__(
    *,
    default_model: str | BaseChatModel,
    default_tools: Sequence[BaseTool | Callable | dict[str, Any]] | None = None,
    default_middleware: list[AgentMiddleware] | None = None,
    default_interrupt_on: dict[str, bool | InterruptOnConfig] | None = None,
    subagents: list[SubAgent | CompiledSubAgent] | None = None,
    system_prompt: str | None = TASK_SYSTEM_PROMPT,
    general_purpose_agent: bool = True,
    task_description: str | None = None,
) -> None

```

Initialize the SubAgentMiddleware.

meth `wrap_model_call`

```

wrap_model_call(
    request: ModelRequest, handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse

```

Update the system prompt to include instructions on using subagents.

meth `awrap_model_call` async

```

awrap_model_call(

```

```

    request: ModelRequest, handler: Callable[[ModelRequest],
    Awaitable[ModelResponse]]
) -> ModelResponse

```

(async) Update the system prompt to include instructions on using subagents.

`func` create_deep_agent

```

create_deep_agent(
    model: str | BaseChatModel | None = None,
    tools: Sequence[BaseTool | Callable | dict[str, Any]] | None = None,
    *,
    system_prompt: str | None = None,
    middleware: Sequence[AgentMiddleware] = (),
    subagents: list[SubAgent | CompiledSubAgent] | None = None,
    response_format: ResponseFormat | None = None,
    context_schema: type[Any] | None = None,
    checkpointer: Checkpointer | None = None,
    store: BaseStore | None = None,
    backend: BackendProtocol | BackendFactory | None = None,
    interrupt_on: dict[str, bool | InterruptOnConfig] | None = None,
    debug: bool = False,
    name: str | None = None,
    cache: BaseCache | None = None,
) -> CompiledStateGraph

```

Create a deep agent.

This agent will by default have access to a tool to write todos (`write_todos`), seven file and execution tools: `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`, `execute`, and a tool to call subagents.

The `execute` tool allows running shell commands if the backend implements `SandboxBackendProtocol`. For non-sandbox backends, the `execute` tool will return an error message.

PARAMETER	DESCRIPTION
<code>model</code>	The model to use. Defaults to Claude Sonnet 4. TYPE: <code>str</code> <code>BaseChatModel</code> <code>None</code> DEFAULT: <code>None</code>
<code>tools</code>	The tools the agent should have access to. TYPE: <code>Sequence[BaseTool Callable dict[str, Any]]</code> <code>None</code>

PARAMETER	DESCRIPTION	DEFAULT: <code>None</code>
<code>system_prompt</code>	The additional instructions the agent should have. Will go in the system prompt.	TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>middleware</code>	Additional middleware to apply after standard middleware.	TYPE: <code>Sequence[AgentMiddleware]</code> DEFAULT: <code>()</code>
<code>subagents</code>	The subagents to use. Each subagent should be a dictionary with the following keys: - <code>name</code> - <code>description</code> (used by the main agent to decide whether to call the sub agent) - <code>prompt</code> (used as the system prompt in the subagent) - (optional) <code>tools</code> - (optional) <code>model</code> (either a <code>LanguageModelLike</code> instance or dict settings) - (optional) <code>middleware</code> (list of <code>AgentMiddleware</code>)	TYPE: <code>list[SubAgent CompiledSubAgent]</code> <code>None</code> DEFAULT: <code>None</code>
<code>response_format</code>	A structured output response format to use for the agent.	TYPE: <code>ResponseFormat</code> <code>None</code> DEFAULT: <code>None</code>
<code>context_schema</code>	The schema of the deep agent.	TYPE: <code>type[Any]</code> <code>None</code> DEFAULT: <code>None</code>
<code>checkpointer</code>	Optional checkpointer for persisting agent state between runs.	TYPE: <code>Checkpointer</code> <code>None</code> DEFAULT: <code>None</code>
<code>store</code>	Optional store for persistent storage (required if backend uses <code>StoreBackend</code>).	TYPE: <code>BaseStore</code> <code>None</code> DEFAULT: <code>None</code>
<code>backend</code>	Optional backend for file storage and execution. Pass either a <code>Backend</code> instance or a callable factory like <code>lambda rt: StateBackend(rt)</code> . For execution support, use a backend that implements <code>SandboxBackendProtocol</code> .	

PARAMETER	DESCRIPTION
	TYPE: BackendProtocol BackendFactory None DEFAULT: None
interrupt_on	Optional Dict[str, bool InterruptOnConfig] mapping tool names to interrupt configs. TYPE: dict[str, bool InterruptOnConfig] None DEFAULT: None
debug	Whether to enable debug mode. Passed through to create_agent. TYPE: bool DEFAULT: False
name	The name of the agent. Passed through to create_agent. TYPE: str None DEFAULT: None
cache	The cache to use for the agent. Passed through to create_agent. TYPE: BaseCache None DEFAULT: None

RETURNS	DESCRIPTION
CompiledStateGraph	A configured deep agent.